

CMPS4191 • LABORATORY 2

Measuring an Asynchronous Report API

An empirical comparison of acknowledgment time and completion time

Duration	Work mode	Code
75 minutes	Pairs	Provided

Central question: Does the asynchronous contract keep the acknowledgment time independent of the report-generation duration?

Learning outcomes

- Measure acknowledgment time separately from completion time.
- Trace a submitted request through the queued, processing, and terminal job states.
- Compare asynchronous measurements with the synchronous results from Lab 1.
- Evaluate whether the new contract satisfies its stated requirements.
- Avoid the incorrect conclusion that asynchronous execution makes work faster.

The revised contract

POST /v1/reports no longer promises the completed report. It promises to acknowledge accepted work and return a stable job identity.

POST promise 202 Accepted means the report request was accepted for processing. It does not mean that the report generation has completed or succeeded.

The client later uses GET /v1/jobs/{id} to observe queued, processing, completed, or failed jobs and to retrieve the result when available.

Experimental design

Research question

How does artificial report-generation delay affect (1) POST acknowledgment time and (2) total job-completion time in the asynchronous API?

Hypothesis — complete before testing

If the report-generation delay increases, acknowledgment time will remain constant. In contrast, completion time will increase proportionally because the POST handler queues the job and immediately returns 202, while a separate background worker performs the delayed report work.

Variables

Role	Variable
Independent	Artificial worker delay: 0 s, 3 s, 7 s, 12 s
Dependent	POST acknowledgment time, total completion time, poll count, observed states, and final state
Controlled	Endpoint, payload, consumer, database, worker count, polling interval, client script,t, and server configuration

Two clocks

Measure	Start	Stop	What it tests
T_ack	Immediately before POST	20responsesse received	Whether the submission waits for the report work
T_complete	Immediately before POST	completed or failed observed	How long does the entire operation take

Key distinction: the new contract should reduce or stabilize T_ack. It does not necessarily reduce T_complete.

Before you measure: predict

Record predictions before running any condition. Use the synchronous evidence from Lab 1, but do not automatically copy its conclusion—the contract has changed.

Delay	Predicted T_ack	Predicted T_complete	Expected final state
0 s	acknowledged	completed	completed
3 s	acknowledged	completed	completed
7 s	acknowledged	completed	completed
12 s	acknowledged	failed	failed

Contract reconstruction

What fact does 202 Accepted communicate?

The request was received and accepted for processing, and a stable job identity was assigned (in the Location header and job_id).

What fact does it not communicate?

That the report has been generated, that processing has been completed, or that it will succeed.

Why must the server return a job identifier?

So the client has a stable handle to later observe status and retrieve the eventual result via GET /v1/jobs/{id}.

Which request retrieves the eventual outcome?

GET /v1/jobs/{public-id} — returns queued, processing, completed, or failed, plus the result when available.

Which component performs the report work?

The background report worker (goroutine in cmd/api/worker.go, processNextReportJob), never the POST handler.

Procedure

Part A — Verify the supplied system

- The POST handler creates a queued job and immediately returns 202 Accepted.
- The worker applies the configured artificial delay, generates the report, and records the result.
- GET /v1/jobs/{id} returns the job's current state.
- The measurement script polls every 250 ms until the measurement completes or fails.

Controlled condition: The -report-delay value must be applied by the worker, not by the POST handler. Delaying the POST handler would recreate the synchronous waiting problem.

Part B — Create request.json

```
{
  "consumer_id": "0198f000-0000-7000-8000-000000000001",
  "from": "2026-01-01T00:00:00Z",
  "to": "2027-01-01T00:00:00Z"
}
```

Part C — Start one condition

```
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=0s
# Repeat later with: 3s, 7s and 12s
```

Measurement script

Save the supplied script as `measure_async.sh`. Read it before running it: identify where `T_ack` starts and ends, where `T_complete` ends, and how the terminal state is detected.

```
#!/usr/bin/env bash
set -u

api='http://localhost:4000/v1'
start_ns=$(date +%s%N)

submit=$(curl --silent --show-error \
  --write-out '%{time_total}' \
  --request POST \
  --header 'Content-Type: application/json' \
  --data @request.json \
  "$api/reports")

ack_time=${submit##*'\n'}
body=${submit%*\n*}
job_id=$(printf '%s' "$body" | jq -r '.job_id')
polls=0

while true; do
  job=$(curl --silent --show-error "$api/jobs/$job_id")
  status=$(printf '%s' "$job" | jq -r '.job.status')
  polls=$((polls + 1))
  printf 'poll=%d status=%s\n' "$polls" "$status"
  case "$status" in completed|failed) break ;; esac
  sleep 0.25
done

end_ns=$(date +%s%N)
complete_time=$(awk -v s="$start_ns" -v e="$end_ns" \
  'BEGIN { printf "%.3f", (e-s)/1000000000 }')
printf 'job_id=%s\nack_time=%ss\ncomplete_time=%ss\npolls=%d\nfinal=%s\n' \
  "$job_id" "$ack_time" "$complete_time" "$polls" "$status"
```

Make it executable and run it:

```
chmod +x measure_async.sh
./measure_async.sh
```

Run the controlled experiment.

1. Run the script three times for the current delay.
2. Record T_ack, T_complete, poll count, and final state.
3. Inspect at least one GET response in each condition.
4. Restart the server with the next delay.
5. Do not run different delay conditions simultaneously.

Condition: 0 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final state	States observed
1	0.004303s	0.048s	1	completed	completed
2	0.016855s	0.074s	1	completed	completed
3	0.015992s	0.314s	2	completed	queued -> completed

Condition: 3 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final state	States observed
1	0.020179s	3.394s	12	completed	queued -> processing -> completed
2	0.025593s	3.183s	11	completed	queued -> processing -> completed
3	0.025075s	3.153s	11	completed	queued -> processing -> completed

Condition: 7 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final state	States observed
1	0.046879s	7.200s	27	completed	queued -> processing -> completed
2	0.038093s	8.545s	27	completed	queued -> processing -> completed
3	0.039034s	7.439s	28	completed	queued -> processing -> completed

Condition: 12 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final state	States observed
1	0.014746s	12.152s	45	completed	queued - > processing - > completed
2	0.036449s	12.402s	46	completed	queued - > processing - > completed
3	0.007999s	12.337s	46	completed	queued - > processing - > completed

Summarize and compare

Calculate the mean for each clock: $\text{mean} = (T_1 + T_2 + T_3) \div 3$.

Delay	Mean T_ack	Mean T_complete	Successful jobs
0 s	0.012383s	0.1453s	/ 3
3 s	0.0236156s	3.243s	/ 3
7 s	0.0413353s	7.728s	3 / 3
12 s	0.0197313s	36.891s	3 / 3

Compare with Lab 1

Delay	Synchronous response time	Async T_ack	Async T_complete
0 s	0.0014s	0.012383s	0.1453s
3 s	3.01s	0.0236156s	3.243s
7 s	7.01s	0.0413353s	7.728s
12 s	12.01s	0.0197313s	36.891s

Observation

Write what changed numerically without explaining the mechanism.

Across all four delay conditions, mean T_{ack} remained within a narrow range of 0.012s to 0.041s, while mean T_{complete} rose in step with the configured delay: 0.145s at 0s, 3.243s at 3s, 7.728s at 7s, and 12.297s at 12s. Poll count also increased with delay, from 1-2 polls at 0s to 45-46 polls at 12s. By contrast, Lab 1's synchronous response time tracked delay in an almost 1:1 ratio (0.0014s, 3.01s, 7.01s, 12.01s), nearly identical to this lab's T_{complete} , but with no equivalent to the flat T_{ack} .

Explanation

Why did T_{ack} behave differently from synchronous response time?

In Lab 1, the client had to wait until the report was fully generated, so the response time increased with the delay. In this lab, the POST handler only adds the job to the queue and returns a 202 response before the report starts generating. Therefore, T_{ack} only measures the time needed to save the job and send the response, not the time needed to generate the report. This is why it remains almost the same even when the delay increases from 0 to 12 seconds.

Why did T_{complete} still change with artificial delay?

T_{complete} changes because the report generator did not disappear; it was moved to a background worker. It measured the time from when the POST request was submitted until the client sees the job as either completed or failed through polling. Therefore, it still includes the full report-generation time, plus some extra time due to the polling interval.

Which component now owns the long-running work?

The background report worker (which processes jobs pulled from the queue) handles report generation, not the HTTP handler that receives the POST request.

What evidence shows that the original request ended before the report was completed?

The evidence is that T_{ack} is consistently much smaller than T_{complete} in every condition. This shows that the POST request finished quickly, while the report was still being processed in the background. The client only found out that the report was completed later through separate GET polling requests.

Evaluate the contract

For each requirement, use your measurements—not intuition—to decide whether the async design provided the property.

Requirement	Evidence	Supported?
Acknowledge without waiting for report completion.	T_ack 0.012–0.041 s across 0–12 s delays; 12 s job acked in 8–36 ms	Yes / No / Unclear
Allow work to continue after POST ends	All 12/12 jobs completed; T_complete up to 12.3 s » T_ack (ms)	Yes / No / Unclear
Expose current job state	Every nonzero condition observed queued → processing → completed via GET	Yes / No / Unclear
Exposes the completed result or failure	Final state completed with retrievable result in every run	Yes / No / Unclear

Limitations and new questions

State two properties this experiment did not test—for example, worker-crash recovery, duplicate submissions, authorization, retention, or multiple-worker contention.

1. Worker-crash/restart recovery — a job left processing when the process dies has no heartbeat/re-queue mechanism and would stay wedged; untested because no crash occurred.
2. Duplicate submissions / idempotency — no idempotency key, so repeated identical POSTs create separate jobs; untested because each run submitted once. (Alternatives: authorization, retention, multiple-worker contention.)

In-class exit ticket

Element	Your response
Claim	Did the async contract decouple acknowledgment from report duration? Yes - the async contract decoupled acknowledgment from report duration.
Evidence	Provide one T_ack comparison and one T_complete comparison.

Element	Your response
	Mean T_ack 0.012 s at 0 s delay vs 0.020 s at 12 s (flat, no trend), while mean T_complete rose from 0.145 s to 12.297 s.
Reasoning	Explain the mechanism that produced the pattern. The POST handler only writes a queued job and returns 202; a background worker polls, applies the delay, generates the report, and marks it completed. The client is freed immediately, but job latency is unchanged.
Limitation	Name one conclusion the experiment cannot support. Cannot conclude async makes work faster, T_complete still grew with the delay, tested with one worker, no crashes, single client.

Complete outside of class

Submit a short comparative laboratory report containing:

- Research question and preregistered hypothesis
- Variables, controls, and polling interval
- Raw asynchronous measurements and calculated means
- A comparison with the synchronous results from Lab 1
- Observation separated from causal explanation
- A requirement-by-requirement contract evaluation
- At least two experimental limitations
- One next experiment that would test an unresolved property

Required reasoning chain

Layer	Question
Evidence	What did T_ack and T_complete do as work duration increased? Mean T_ack stayed flat with no upward trend (0.012, 0.024, 0.041, 0.020 s for 0→12 s delays), while mean T_complete rose with the delay (0.145, 3.243, 7.728, 12.297 s).
Requirement	Which required property was tested? We tested the decoupling property that T_ack must remain independent of the duration of report generation.

Layer	Question
Contract	What did POST promise, and what did GET promise? POST promised only a 202 Accepted request for processing, plus a stable job ID (Location header). GET promised the current state (queued/processing/completed/failed) and the result when it is available.
Implementation	Which mechanism produced the observed behavior? The background worker (cmd/api/worker.go) produced the observed behavior claiming jobs via SKIP LOCKED, applying the configured delay, generating the report, and marking it completed.
Evaluation	Did the evidence support the contract claim? Yes, it did. T_ack stayed approximately between 0.012–0.041 s across all four delays, while T_complete scaled with the delay;

Final distinction: Asynchronous execution can make the client free sooner without making the report finish sooner. The experiment must report both facts.